

## BÀI 05: LÀM VIỆC VỚI DATABASE TRONG NESTJS THÔNG QUA TYPEORM

**Mục tiêu bài 05:** Bài này sẽ giới thiệu đến các bạn các vấn đề về TypeORM, cài đặt và cấu hình kết nối dữ liệu vào Nestjs thông qua TypeORM. Giới thiệu các bạn về các hàm cơ bản thao tác xử lý dữ liệu MySQL trong Nestjs thông qua TypeORM.

### 5.1. TỔNG QUAN VỀ TYPEORM

**TypeORM** là một **ORM** (Object Relational Mapping) cho phép tương tác với cơ sở dữ liệu thông qua các **class** và **object** thay vì viết trực tiếp các câu lệnh **SQL**.

Hỗ trợ nhiều loại cơ sở dữ liệu như: **PostgreSQL, MySQL, SQLite, SQL Server,...**

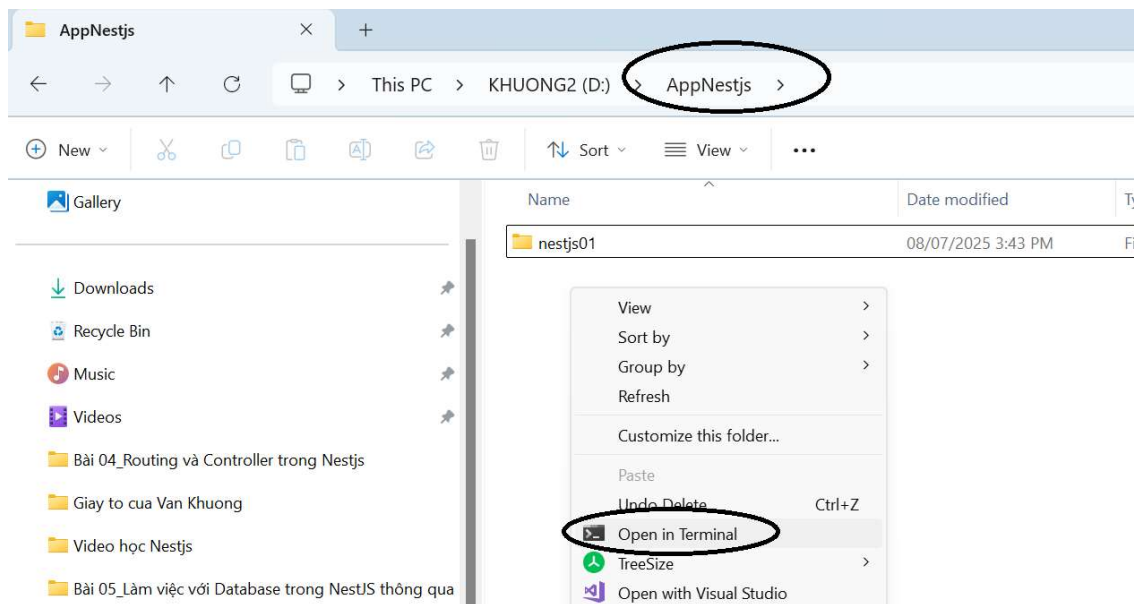
**Lợi ích của TypeORM:**

- Tự động ánh xạ bảng (**table**) vào các entity (**class**).
- Dễ dàng thực hiện **CRUD** mà không cần viết **SQL** thủ công.
- Tương thích tốt với **Nestjs**.

### 5.2. CÀI ĐẶT TYPEORM CHO DỰ ÁN NESTJS

**Ví dụ 5.1:** Cài đặt TypeORM trong dự án Nestjs

- Đầu tiên các bạn tạo dự án **Bai05** trong thư mục **AppNestjs** đã tạo ở bài trước, bằng cách: click chuột phải vào vùng trống của thư mục đó, chọn **Open in Terminal**



- Xuất hiện giao diện **cmd**, các bạn gõ lệnh: **nest new Bai05** (để tạo dự án Bai05). Chọn **npm**, tiếp tục bấm **Enter**. Chờ cho quá trình tạo dự án diễn ra và báo quá trình tạo dự án thành công.

```
Windows PowerShell
PS D:\AppNestjs> nest new Bai05
We will scaffold your app in a few seconds..

Which package manager would you ❤️ to use? npm
CREATE bai05/.prettierrc (54 bytes)
CREATE bai05/.eslint.config.mjs (869 bytes)
CREATE bai05/nest-cli.json (179 bytes)
CREATE bai05/package.json (2104 bytes)
CREATE bai05/README.md (5126 bytes)
CREATE bai05/tsconfig.build.json (101 bytes)
CREATE bai05/tsconfig.json (565 bytes)
CREATE bai05/src/app.controller.ts (286 bytes)
CREATE bai05/src/app.module.ts (259 bytes)
CREATE bai05/src/app.service.ts (150 bytes)
CREATE bai05/src/main.ts (236 bytes)
CREATE bai05/src/app.controller.spec.ts (639 bytes)
CREATE bai05/test/jest-e2e.json (192 bytes)
CREATE bai05/test/app.e2e-spec.ts (699 bytes)

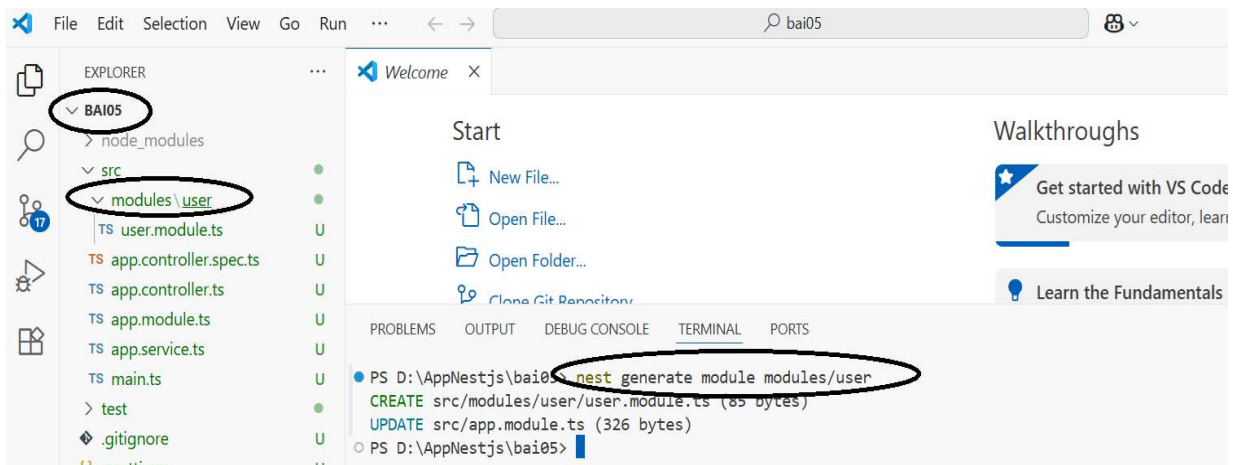
Installation in progress...

Successfully created project bai05
Get started with the following commands:

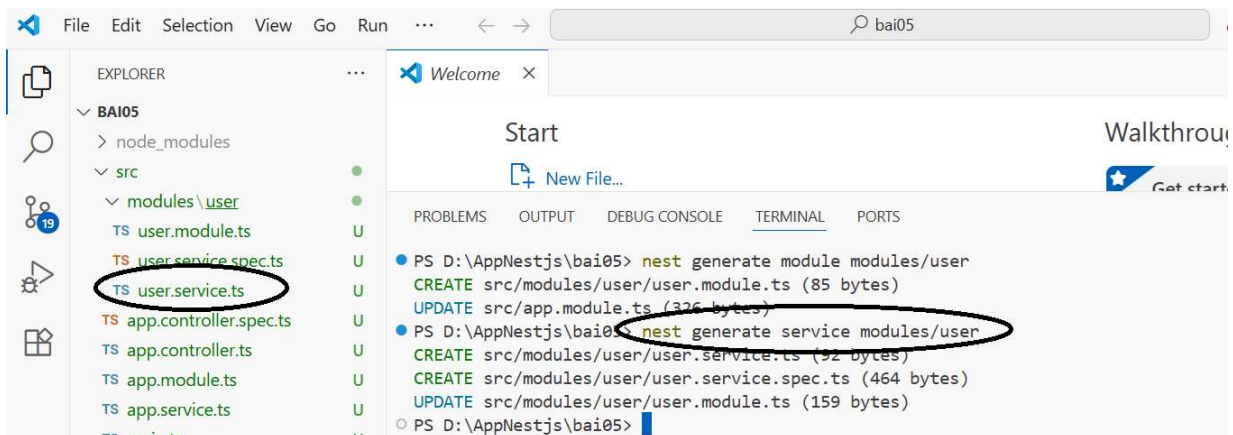
$ cd bai05
$ npm run start
```

- Mở chương trình VSCode lên, mở thư mục dự án **Bai05**

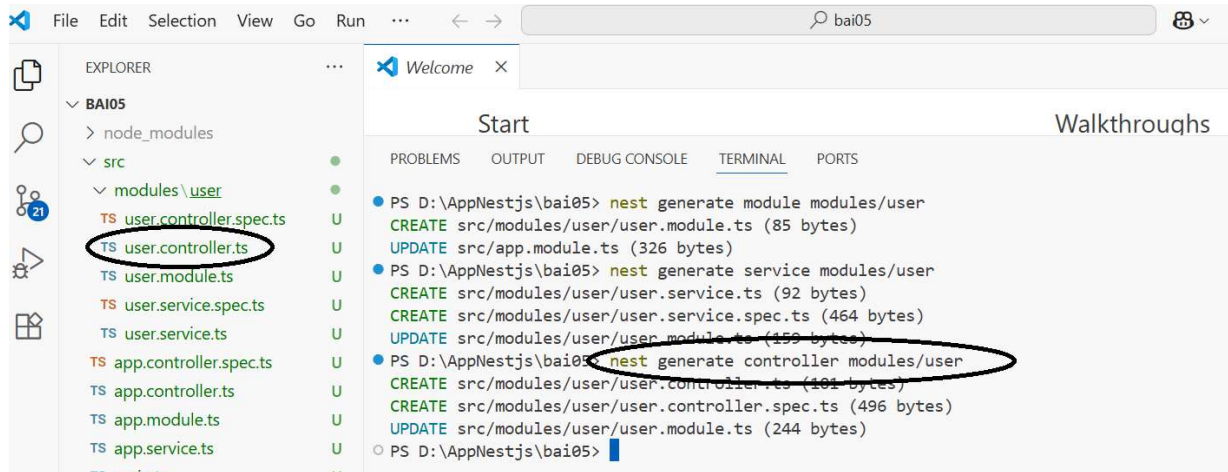
- Các bạn mở **Terminal** của VScode gõ dòng lệnh: **nest generate module modules/user**.  
Đợi một lát sẽ thấy một **module user** tạo ra trong dự án (nằm trong thư mục **modules**).



- Ở Terminal các bạn gõ dòng lệnh: **nest generate service modules/user** để tạo service.  
Đợi 1 lát sẽ tạo ra tập tin **user.service.ts**



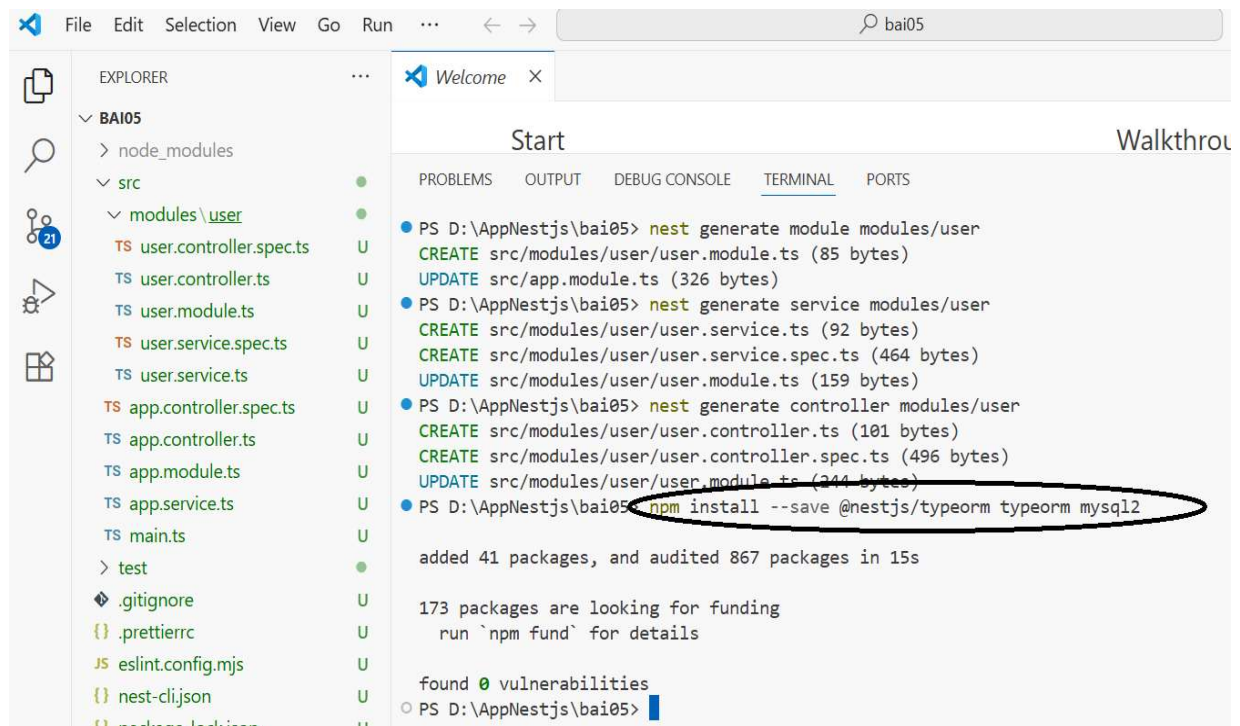
- Tiếp tục ở Terminal các bạn gõ dòng lệnh: **nest generate controller modules/user** để tạo controller. Dự 1 lát sẽ tạo ra tập tin **user.controller.ts**



- Tiếp tục, các bạn Cài đặt **TypeORM** vào dự án **Bai05** bằng cách tại Terminal gõ dòng lệnh: **npm install --save @nestjs/typeorm typeorm mysql2**

*Giải thích lệnh:*

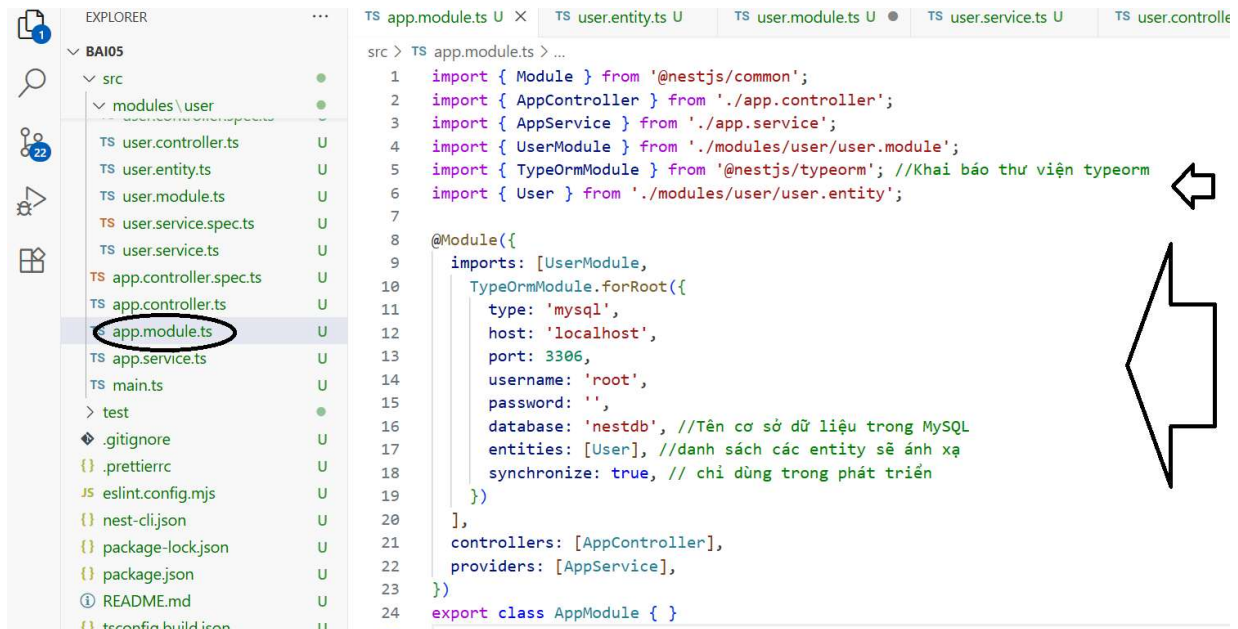
- **@nestjs/typeorm**: module typeorm cho Nestjs
- **typeorm**: thư viện typeorm
- **mysql2**: Driver cho mysql (nếu dùng **PostgreSQL** thì thay bằng **pg**)



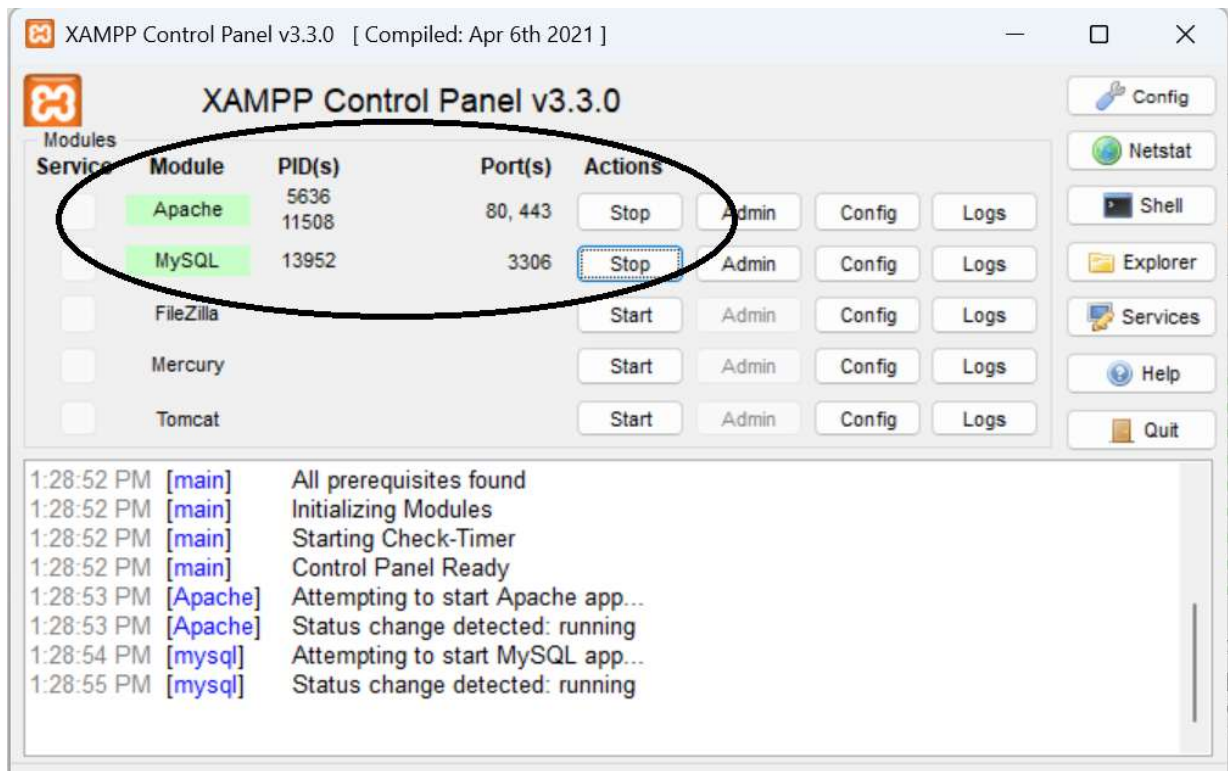
- Như vậy các bạn đã tạo dự án **Nestjs** và cài đặt **TypeORM** vào dự án.

### 5.3. CẤU HÌNH KẾT NỐI TYPEORM KẾT NỐI CƠ SỞ DỮ LIỆU MYSQL

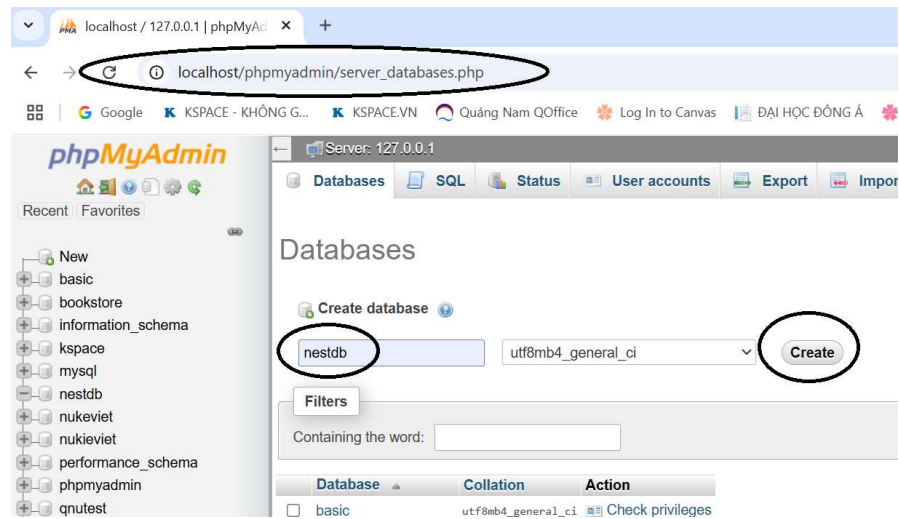
- Các bạn mở file **app.module.ts** và viết code như bên dưới:



- Chương trình sử dụng cơ sở dữ liệu **MySQL** nên các bạn có thể tải và cài đặt phần mềm **Xampp**. Chạy chương trình **Xampp** lên và chạy 2 dịch vụ **Apache** và **MySQL** như bên dưới: (Nếu bị lỗi khi chạy 2 dịch vụ này có thể do xung đột cổng, các bạn có thể tìm hướng dẫn xử lý trên Internet)



- Tiếp tục, các bạn tạo một cơ sở dữ liệu tên là **nestdb** bằng cách mở trình duyệt Website (ví dụ: Chrome, ...) gõ đường dẫn: **localhost/phpmyadmin/**, chọn vào thẻ **Database**, gõ tên cơ sở dữ liệu là: **nestdb** và chọn **Create** như bên dưới:



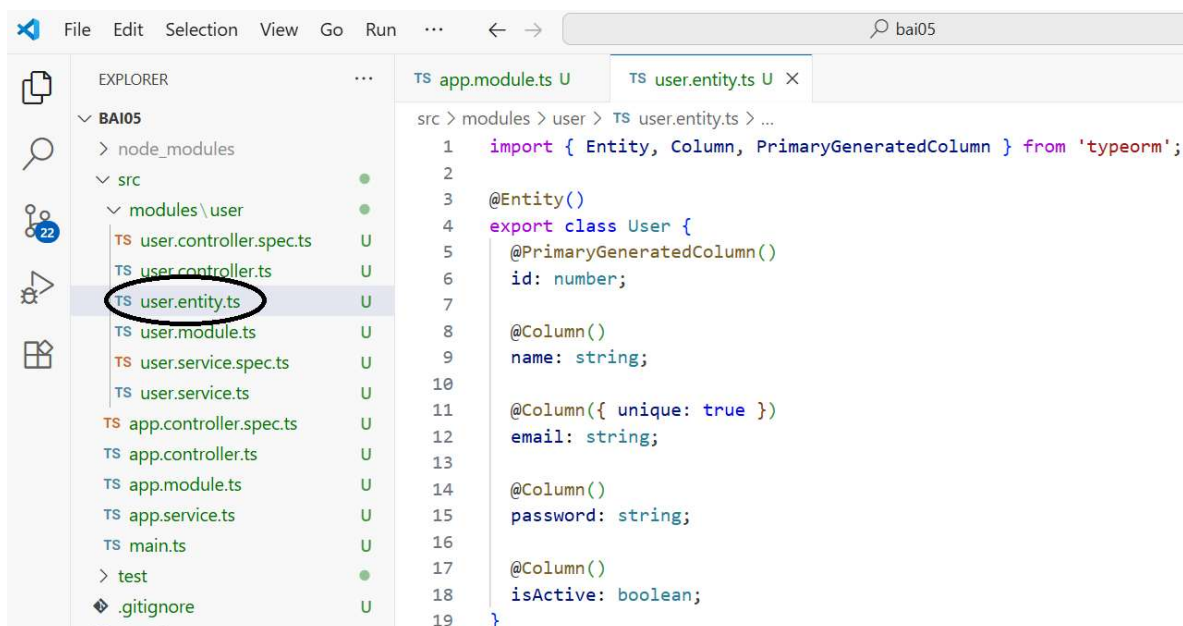
## 5.4. ĐỊNH NGHĨA ENTITY

**Entity** là lớp (**class**) đại diện cho một bảng (**table**) trong cơ sở dữ liệu.

Mỗi thuộc tính (**property**) trong lớp sẽ tương ứng với một cột (**column**) trong bảng.

Entity là cầu nối giữa ứng dụng và cơ sở dữ liệu, dễ dàng thao tác với bảng thông qua các hàm của **TypeORM** như: **find**, **save**, **update**, **delete**... Là một phần của kiến trúc **Model** trong mô hình **MVC** (Model-View-Controller).

- Để tạo Entity cho ứng dụng đang làm, các bạn tiếp tục tạo trong thư mục **user** tập tin **user.entity.ts** và viết code như sau:



## 5.5. IMPORT ENTITY VÀO MODULE

- Để Import Entity cho ứng dụng đang làm, các bạn tiếp tục mở tập tin **user.module.ts** và viết code như sau:

```

1 // user.module.ts
2 import { Module } from '@nestjs/common';
3 import { TypeOrmModule } from '@nestjs/typeorm';
4 import { User } from './user.entity';
5 import { UserService } from './user.service';
6 import { UserController } from './user.controller';
7
8 @Module({
9   imports: [TypeOrmModule.forFeature([User])], // Import Entity
10  providers: [UserService],
11  controllers: [UserController],
12 })
13 export class UserModule {}

```

## 5.6. REPOSITORY TRONG TYPEORM

**Repository** là nơi thực hiện các thao tác với Database như **CRUD** (Create, Read, Update, Delete).

- Bây giờ các bạn thực hiện **Inject Repository** vào **Service** và tạo các hàm cơ bản xử lý cơ sở dữ liệu bằng cách: mở tập tin **user.service.ts** và viết code như bên dưới:

```

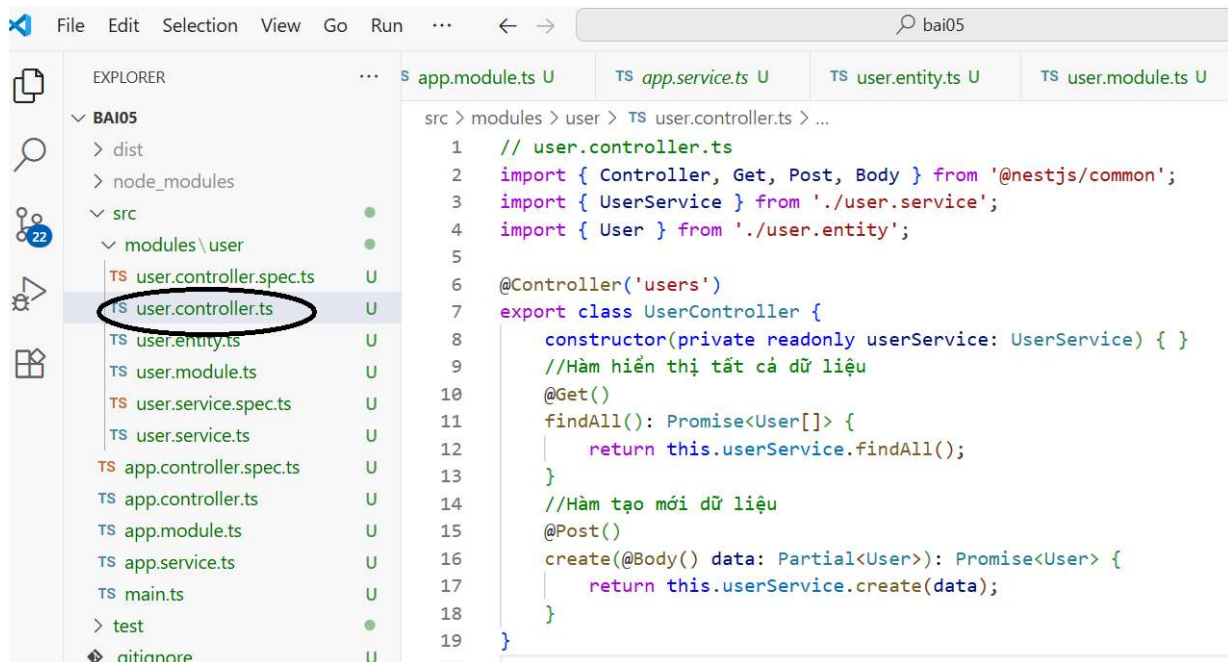
1 // user.service.ts
2 import { Injectable } from '@nestjs/common';
3 import { InjectRepository } from '@nestjs/typeorm';
4 import { Repository } from 'typeorm';
5 import { User } from './user.entity';
6
7 @Injectable()
8 export class UserService {
9   constructor(
10     //Inject Repository vào Service
11     @InjectRepository(User)
12     private readonly userRepository: Repository<User>, // Inject repository
13   ) {}
14   //Tạo hàm tìm kiếm tất cả dữ liệu
15   async findAll(): Promise<User[]> {
16     return this.userRepository.find();
17   }
18   //Tạo hàm tạo cơ sở dữ liệu
19   async create(userData: Partial<User>): Promise<User> {
20     const user = this.userRepository.create(userData);
21     return this.userRepository.save(user);
22   }
23 }

```

- Như vậy chúng ta đã xây dựng được 2 hàm xử lý dữ liệu là: **Hàm tìm và hiển thị** tất cả dữ liệu, **hàm tạo mới** dữ liệu.

## 5.7. CONTROLLER GỌI SERVICE ĐỂ THỰC HIỆN CRUD

- Để gọi **Service** để thực hiện 2 hàm (phương thức) với cơ sở dữ liệu đã tạo các bạn mở tập tin **user.controller.ts** và viết code như bên dưới:

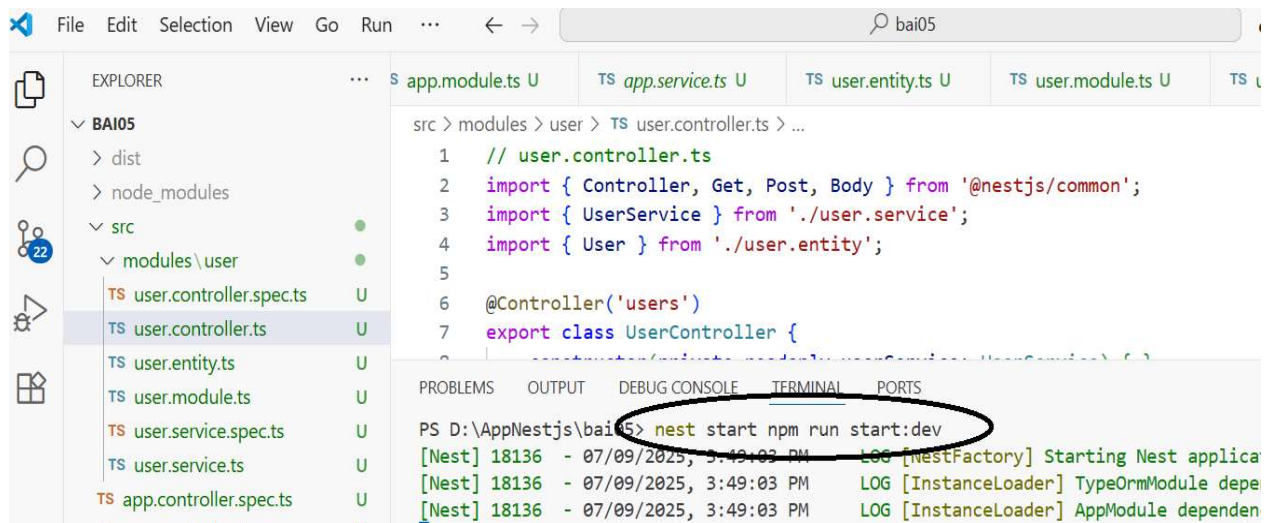


```

1 // user.controller.ts
2 import { Controller, Get, Post, Body } from '@nestjs/common';
3 import { UserService } from './user.service';
4 import { User } from './user.entity';
5
6 @Controller('users')
7 export class UserController {
8   constructor(private readonly userService: UserService) { }
9   //Hàm hiển thị tất cả dữ liệu
10  @Get()
11  findAll(): Promise<User[]> {
12    return this.userService.findAll();
13  }
14  //Hàm tạo mới dữ liệu
15  @Post()
16  create(@Body() data: Partial<User>): Promise<User> {
17    return this.userService.create(data);
18  }
19 }
  
```

- Như vậy 2 hàm xử lý dữ liệu đã được đưa vào **Controller**.

- Bây giờ các bạn mở **Terminal** và biên dịch chương trình: **nest start npm run start:dev**

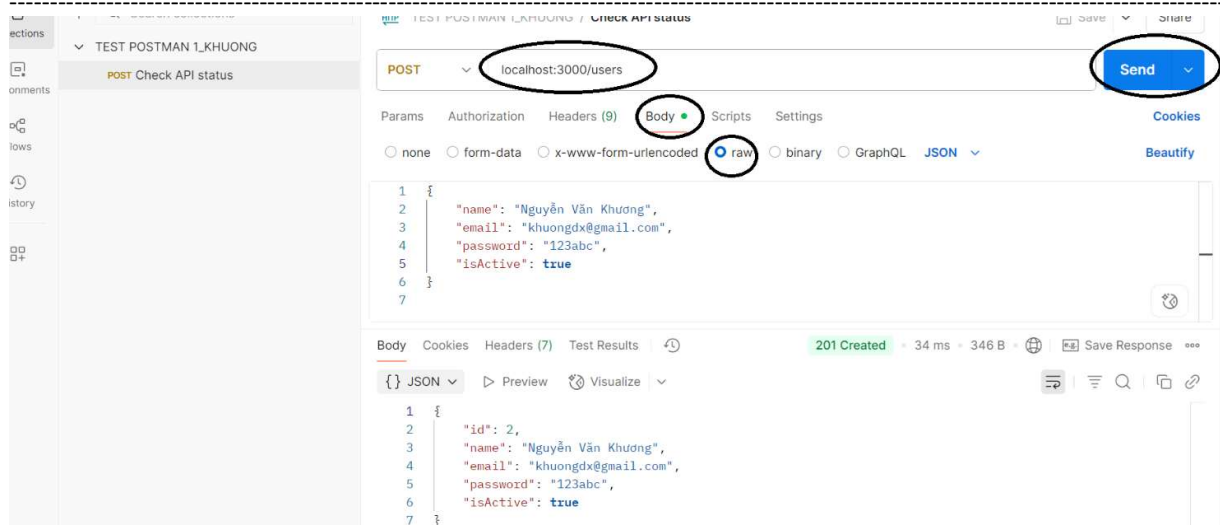


```

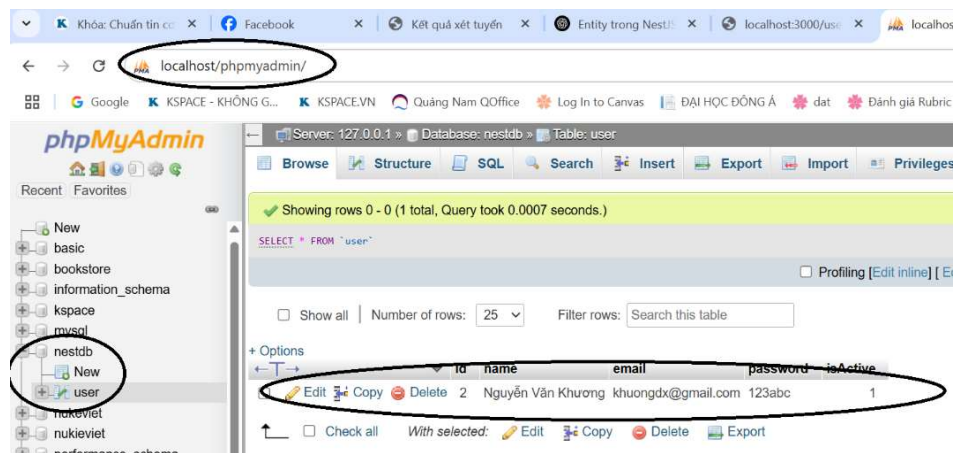
PS D:\AppNestjs\bai05> nest start npm run start:dev
[Nest] 18136 - 07/09/2025, 3:49:03 PM LOG [NestFactory] Starting Nest applica
[Nest] 18136 - 07/09/2025, 3:49:03 PM LOG [InstanceLoader] TypeOrmModule depe
[Nest] 18136 - 07/09/2025, 3:49:03 PM LOG [InstanceLoader] AppModule dependen
  
```

- Để kiểm tra chương trình chạy thế nào các bạn mở chương trình **Postman** lên, chọn phương thức **POST**, gõ đường dẫn: **localhost:3000/users**. Chọn thẻ **Body**, chọn **Raw**, gõ nội dung dữ liệu cần thêm vào và chọn **Send**, sẽ thấy kết quả bên dưới:

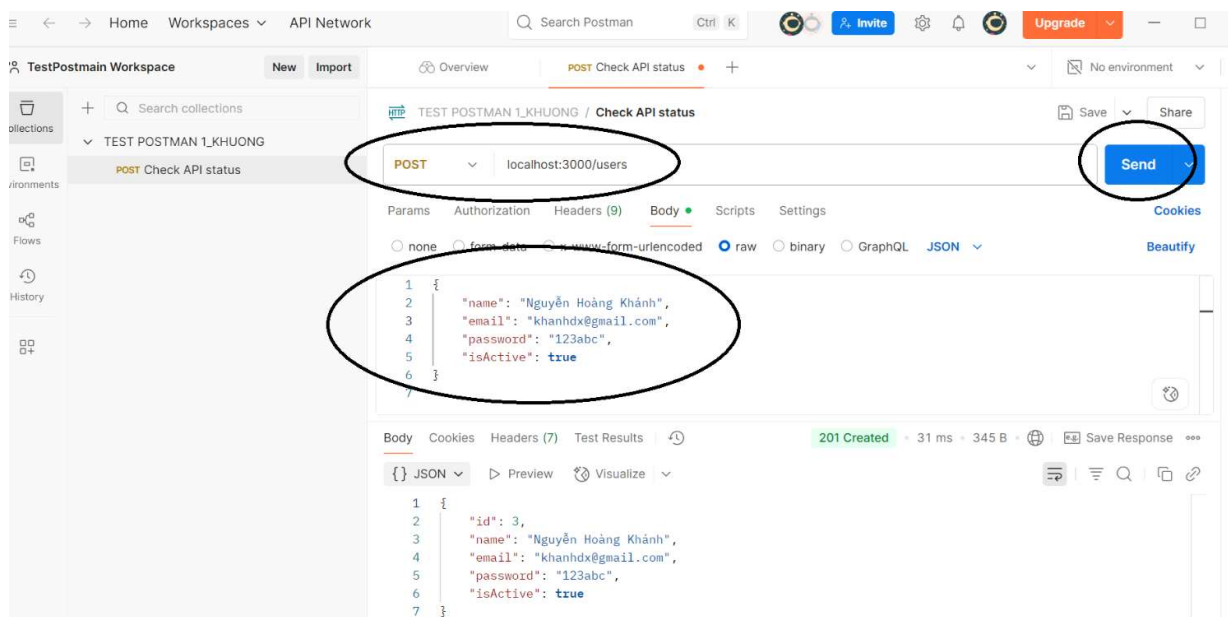
## Bài 05: Làm việc với Database trong NestJS thông qua TypeORM



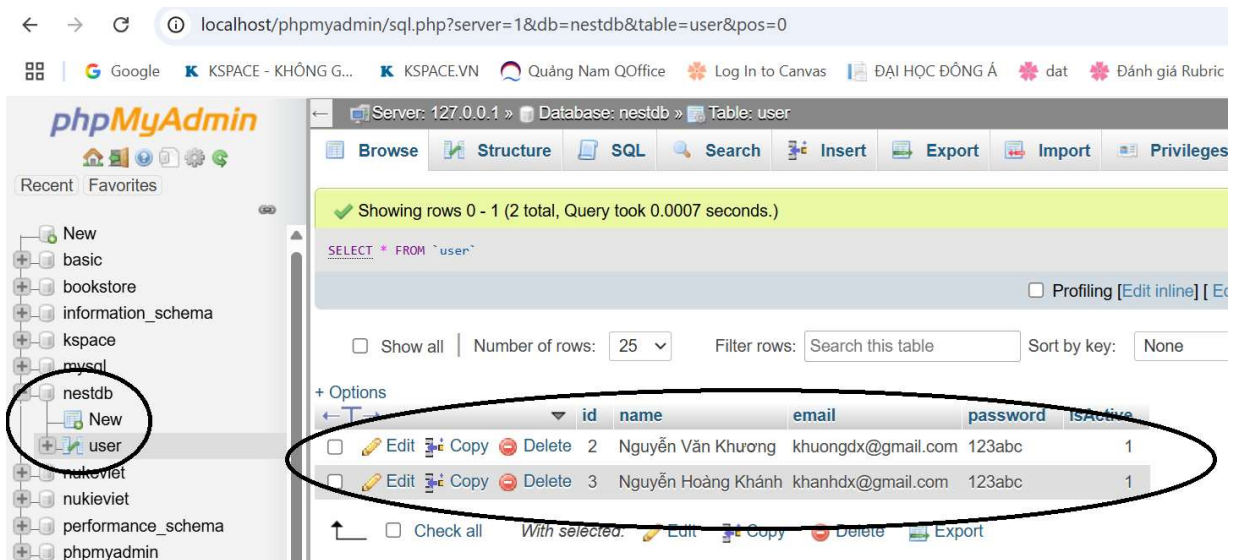
- Mở lại cơ sở dữ liệu sẽ thấy dữ liệu được thêm vào bảng **User** trong Cơ sở dữ liệu **nestdb**



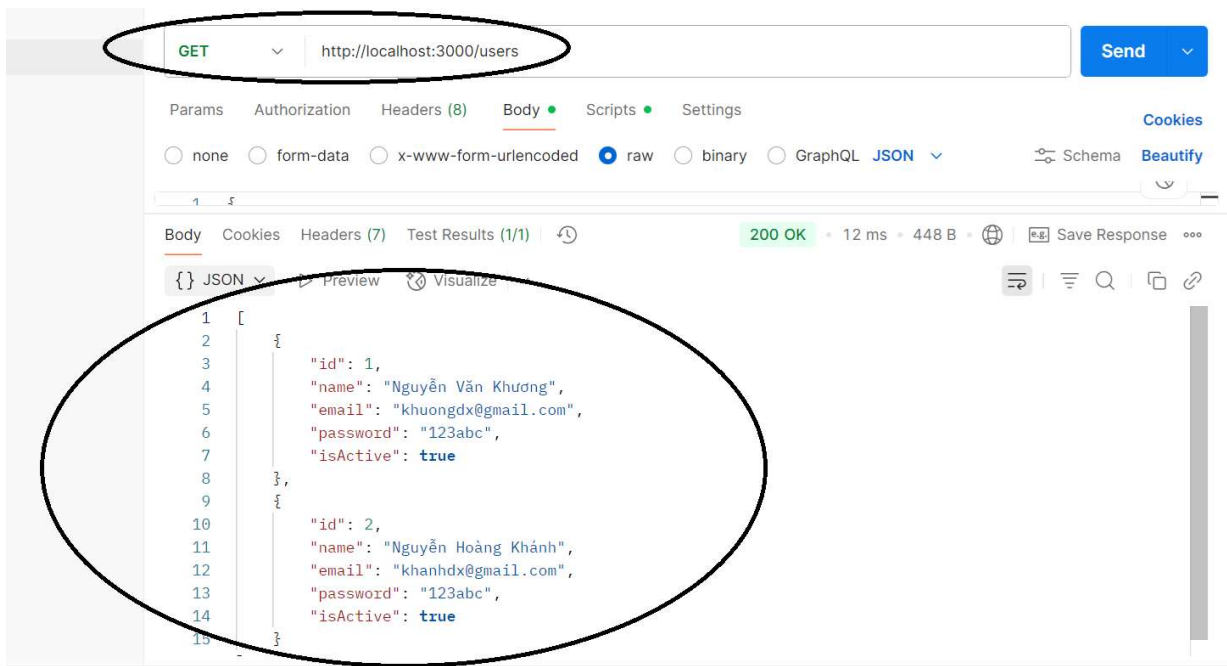
- Tiếp tục ở **Postman** các bạn tạo thêm 1 bản ghi dữ liệu nữa như sau:



- Các bạn xem lại trong cơ sở dữ liệu **Mysql** đã có 2 bản ghi dữ liệu được tạo ra:



- Thực hiện kiểm tra hàm hiển thị tất cả của ứng dụng, các bạn mở **Postman** chọn phương thức **GET** và cấu hình như bên dưới sẽ hiển thị tất cả dữ liệu:



- Như vậy các bạn đã xây dựng được một ứng dụng **Nestjs** có 2 hàm cơ bản: Hiển thị dữ liệu và Thêm dữ liệu vào Cơ sở dữ liệu MySQL.

- Chúng ta tiếp tục tìm hiểu các hàm xử lý dữ liệu tiếp theo ở phần sau.